



PDF Download
3773274.3774256.pdf
31 March 2026
Total Citations: 0
Total Downloads: 185

 Latest updates: <https://dl.acm.org/doi/10.1145/3773274.3774256>

RESEARCH-ARTICLE

An Autonomic Computing Approach for Scaling Cloud-Based Smart City Platforms

ANDRÉ SOLINO, Federal University of Rio Grande do Norte, Natal, RN, Brazil

THAIS BATISTA, Federal University of Rio Grande do Norte, Natal, RN, Brazil

EVERTON CAVALCANTE, Federal University of Rio Grande do Norte, Natal, RN, Brazil

Open Access Support provided by:

Federal University of Rio Grande do Norte

Published: 31 December 2025

Citation in BibTeX format

UCC '25: 2025 IEEE/ACM 18th
International Conference on Utility and
Cloud Computing
December 1 - 4, 2025
France, France

Conference Sponsors:
SIGARCH

An Autonomic Computing Approach for Scaling Cloud-Based Smart City Platforms

André Solino
Federal University of Rio Grande do
Norte
Natal, Brazil
andre.solino@imd.ufrn.br

Thais Batista
Federal University of Rio Grande do
Norte
Natal, Brazil
thaisbatista@gmail.com

Everton Cavalcante
Federal University of Rio Grande do
Norte
Natal, Brazil
everton.cavalcante@ufrn.br

Abstract

Smart city platforms often use cloud-based infrastructures that leverage containers and virtual machines to manage dynamic demands from diverse data sources, urban services, and users. However, traditional auto-scaling mechanisms in cloud environments typically focus on infrastructure-level metrics, which can result in delayed responses to performance bottlenecks. This paper presents a reactive auto-scaling approach based on the MAPE-K autonomic control loop that integrates infrastructure-level data with application-level metrics. Our approach enables the earlier identification of hotspots in the platform's internal logic, which are often overlooked by traditional techniques. Our solution collects multi-level metrics in a non-intrusive manner to determine whether to scale resources vertically or horizontally before significant resource contention arises. We validated our approach in a real-world smart city platform that manages geospatial data and experiences highly variable workloads. Experimental results show that our solution can reduce reaction times and prevent unnecessary scaling actions.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; *Availability*.

Keywords

Smart cities, Auto-scaling, Cloud computing, MAPE-K

ACM Reference Format:

André Solino, Thais Batista, and Everton Cavalcante. 2025. An Autonomic Computing Approach for Scaling Cloud-Based Smart City Platforms. In *2025 IEEE/ACM 18th International Conference on Utility and Cloud Computing (UCC '25)*, December 01–04, 2025, Nantes, France. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773274.3774256>

1 Introduction

Smart cities leverage Information and Communication Technology (ICT) infrastructures to enhance urban services and improve the well-being of their citizens. By integrating data from many sources, such as sensors, mobile devices, and social networks, *smart*

city platforms can provide valuable insights and services in areas like transportation, public safety, and energy management. The effectiveness of these platforms stems from their ability to manage data from various applications, multiple users, and several interconnected devices and systems.

Cloud computing offers the elasticity required to handle the dynamic workloads of smart city platforms. However, the auto-scaling mechanisms they utilize often fall short as they typically rely on infrastructure-level metrics, such as CPU or RAM utilization, which may not accurately reflect the platform's performance. If an internal bottleneck occurs, e.g., a slow query execution within the platform, the system may only detect the issue once CPU usage increases significantly. While the majority of research addresses scaling strategies in isolation (either purely vertical or horizontal), some works have explored hybrid approaches [9, 13, 18]. Nonetheless, they focus solely on CPU resource scaling, limit their approach to virtual machines (VMs), or address only container scaling.

To overcome these limitations, we present a reactive auto-scaling approach for cloud-based smart city platforms. Our approach is grounded in autonomic computing principles and employs the Monitor-Analyze-Plan-Execute-Knowledge (MAPE-K) control loop [8], a well-established model for self-adaptive systems. In our approach, the auto-scaling mechanism collects the current state of the underlying infrastructure and the platform execution and analyzes this information to determine, based on predefined rules and thresholds, whether scaling is necessary to respond to real-time workloads or resource usage [5]. Our approach concerns vertical and horizontal scaling actions, preventing performance degradation even before significantly impacting infrastructure metrics.

Our key contribution is reducing reaction time by integrating infrastructure-level data from virtual machines (VMs) and containers with application-level metrics, particularly the execution time of internal operations within the smart city platform. A significant advantage of our approach is its ability to monitor the execution time of the platform's internal operations in a non-intrusive manner, meaning it does not interfere with the monitoring or the code itself. We implemented our solution as a modular, extensible architecture and validated it on a real-world smart city platform, demonstrating its ability to reduce reaction times and improve overall Quality of Service (QoS).

The remainder of this paper is structured as follows. Section 2 introduces our reactive auto-scaling approach. Section 3 describes the implementation aspects of our solution. Section 4 reports a performance evaluation of the approach on the computational environment supporting a real-world smart city platform. Section 5

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
UCC '25, Nantes, France

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2285-1/25/12
<https://doi.org/10.1145/3773274.3774256>

discusses related work. Section 6 presents conclusions and outlines future research directions.

2 Reactive Auto-Scaling Approach

Our approach is designed as a self-adaptation mechanism based on the MAPE-K model, allowing the system to monitor its behavior and adjust accordingly to meet performance goals. MAPE-K operates in four phases in a continuous feedback cycle:

- (1) *Monitor* (M): It continuously observes the system and its environment to detect changes or anomalies.
- (2) *Analyze* (A): It evaluates the monitored data to determine if any adaptation is required.
- (3) *Plan* (P): It builds an action plan aimed at optimizing the system's performance based on the identified needs.
- (4) *Execute* (E): It applies the planned changes to the system.

A shared *Knowledge* base (K) maintains essential system details, such as monitoring logs, thresholds, and current resource configurations. This structure ensures that each phase can update and query the most recent view of the system state.

In our auto-scaling scenario, the *Monitor* and *Execute* phases operate directly on the infrastructure (hosts, containers) while also considering application-level operations. When monitoring identifies threshold violations, referred to as *symptoms*, these are passed to the *Analyze* phase for further evaluation. If the analysis concludes that a scaling action is necessary, it sends a *change request* to the *Plan* phase, which creates a *change plan* for the *Execute* phase to carry out. All phases continuously update and query the *Knowledge* base, ensuring a closed loop of feedback and adaptation. The adaptations include vertical scaling, such as adding CPU or memory to a VM, and horizontal scaling, which involves adding or removing container instances to handle workload increases or decreases.

Figure 1 illustrates the proposed auto-scaling process. We implement each phase of the MAPE-K loop through the activities described in this section.

2.1 Monitor

The monitoring phase is responsible for collecting data from three distinct levels:

- *Host level*: It gathers metrics from the underlying physical machines or VMs (host), such as CPU and memory utilization.
- *Container level*: It collects performance data from containers running the smart city platform's services, including CPU and memory utilization per container.
- *Platform level*: It monitors application-specific metrics, including the execution time of internal operations (e.g., queries or service endpoints) within the platform.

Integrating metrics across multiple levels (host, container, and platform) provides a comprehensive view of the system's state and enables reactive adaptations when anomalies are detected. Moreover, by monitoring the platform's execution, we can detect problems that have not yet significantly affected infrastructure metrics. We implemented this monitoring in a non-intrusive manner thanks to aspect-oriented programming (AOP) [11], which enables intercepting function calls without interfering with the main code.

Algorithm 1 Symptom detection procedure

```

1: procedure SYMPTOMDETECTION(config)
2:   clusterNodes  $\leftarrow$  Monitoring.getClusterNodes
3:   deployments  $\leftarrow$  Monitoring.getDeployments
4:   clusterSymptomDetection(config, clusterNodes)
5:   for each deployment  $\in$  deployments
6:     containers  $\leftarrow$  d.getContainers
7:     containerSymptomDetection(config, d, containers)
8:     platformSymptomDetection(config, d, containers)
9:   end for
10:  sendSymptom(symptom)
11: end procedure

```

Monitoring agents associated with each level collect metrics and store them in the *Knowledge* base. A *monitoring manager* receives and organizes the metrics collected by the monitoring agents and sends them to a database. By structuring data at different layers, we can detect mismatches, such as high CPU usage at the container level but normal usage at the host level, or slow platform operations despite low CPU usage, which may indicate specialized bottlenecks.

The core of the *Monitor* phase is the *symptomDetection* procedure (see Algorithm 1). The *config* parameter (line 1) has two main attributes: *updateFrequency* is the time interval at which environment monitoring metrics are periodically collected, and *historyInterval* denotes the temporal duration of the historical data retrieved by a single execution of the symptom detection process. For example, *updateFrequency* = 10 means performing data collection every ten seconds, while *historyInterval* = 120 means the symptom detection process will retrieve monitoring data for the past two minutes.

The *SymptomDetection* procedure detects symptoms by checking static thresholds at the host, container, and platform levels. The *ClusterSymptomDetection* (line 4), *ContainerSymptomDetection* (line 7), and *PlatformSymptomDetection* (line 8) procedures retrieve the monitored items at the desired level (i.e., VM or cluster node, container, or platform item) and perform a threshold-based check on each item. The detected symptoms are sent to the *Analysis* phase.

2.2 Analyze

The *Analyze* phase continuously checks for threshold violations in the monitored metrics. When a metric exceeds a predefined threshold (i.e., a resource violation occurs), it is flagged as a *symptom*. A resource violation as *UP* indicates the need to add more resources, while a resource violation as *DOWN* suggests eliminating resources. These decisions also account for data from different levels (host, container, and platform), providing a more holistic view of the environment. Adding or removing resources is only defined after using the heat map algorithm, as explained later in this paper.

Algorithm 2 describes the core of the decision-making procedure. The first action is to create a change request based on the symptom sent by the *Monitor* phase. Next, the *scalingHeat* procedure calculates the heat map for every change request. The *SendChangeRequest* procedure checks if the *heatThreshold* value has been reached to send a change request to the *Plan* phase.

A common pitfall in reactive approaches is frequent scaling, where transient spikes can cause the system to oscillate between

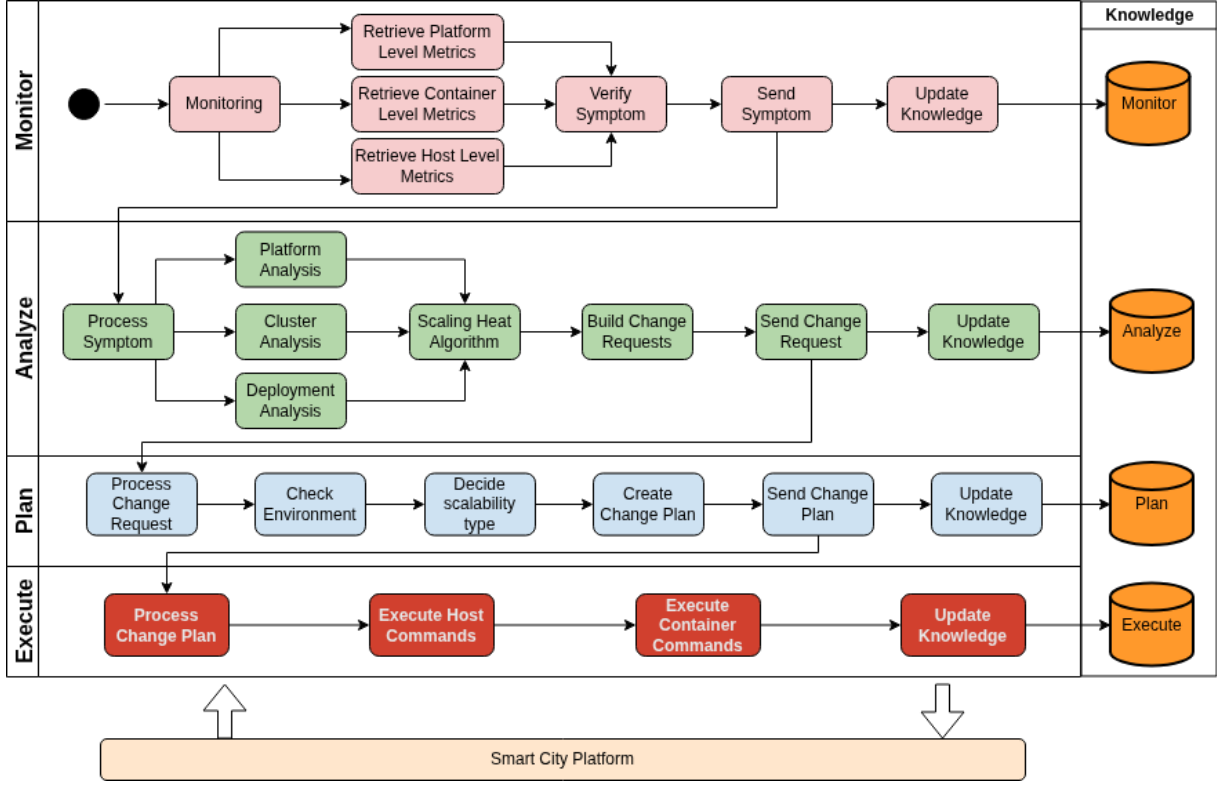


Figure 1: Overview of the proposed auto-scaling process

Algorithm 2 Core decision-making procedure

```

1: procedure DECISIONMAKINGANALYSIS(config, symptom)
2:   create CR from symptom
3:   scalingHeat()
4:   sendChangeRequest(config, CR)
5: end procedure

```

adding and removing resources. To prevent the system from becoming unstable, we consider a heat-based strategy that allows, for example, avoiding issuing scaling requests if CPU usage peaks for a short moment but quickly returns to regular operation. Algorithm 3 presents the *ScalingHeat* procedure, which is an adaptation of the method proposed by Barna et al. [3]. The concept of *heat* determines the accumulation of resources to add or remove. If the upper/lower limit is violated, the heat increases/decreases (lines 5-14). When no threshold violation occurs, the heat gradually returns to zero (lines 15-21).

The *heatThreshold* parameter controls the threshold value that the heat variable must meet to trigger the submission of a change request. For instance, if the parameter is set to 3, the change request will only be dispatched when the threshold is violated continuously for the third time. Conversely, if the value is set to 1, the heatmap algorithm will have no impact, and the change request will be sent as soon as the threshold is reached.

The procedure *sendChangeRequest* (Algorithm 4) checks whether a change is necessary for the monitored environment. A change request is sent to the *Plan* phase when the value of the *heat* attribute reaches the threshold value defined by the *heatThreshold* parameter.

2.3 Plan

Based on the diagnosis, the *Plan* phase formulates a scaling strategy (i.e., a change plan). The change plan may range from a simple command to a complex workflow and contains a set of changes to send to the *Execute* phase. If any change is needed in the underlying infrastructure, the *Analyze* phase sends a change request to the *Plan* phase.

The planning component creates a plan with the necessary actions to fulfill the change request received from the *Analyze* phase. It is also responsible for defining the amount and how resources will be added or removed, as well as accepting or rejecting the suggested quantity. For instance, if a change request is made to remove some replicas equal to or greater than the existing quantity, the planning component will not comply with the request. In this case, adhering to the request will result in no replicas being left, which will cause the system to become unavailable. This information is obtained from the *Knowledge* base.

Creating VMs is a slow process, and horizontal elasticity requires additional components, such as load balancers and their reconfiguration. Launching an instance also takes some time during the boot process to become functional. In addition, after creating the VM,

Algorithm 3 Scaling heat algorithm

```

1: procedure SCALINGHEAT
2:   for  $lastCR \in lastAnalysis$ 
3:     for  $currentCR \in currentAnalysis$ 
4:       if  $lastCR.item = currentCR.item$ 
5:         if  $currentCR.ThresholdViolation = UP$ 
6:           if  $currentCR.Heat \leq 0$ 
7:              $currentCR.Heat \leftarrow 1$ 
8:           else
9:              $currentCR.Heat \leftarrow lastCR + 1$ 
10:          end if
11:         else if  $currentCR.ThresholdViolation = DOWN$ 
12:           if  $currentCR.Heat \geq 0$ 
13:              $currentCR.Heat \leftarrow -1$ 
14:           else
15:              $currentCR.Heat \leftarrow lastCR - 1$ 
16:           end if
17:         else if  $currentCR.ThresholdViolation = NONE$ 
18:           if  $currentCR.Heat > 0$ 
19:              $currentCR.Heat \leftarrow lastCR - 1$ 
20:           else if  $currentCR.Heat < 0$ 
21:              $currentCR.Heat \leftarrow lastCR + 1$ 
22:           else
23:              $currentCR.Heat \leftarrow 0$ 
24:           end if
25:         end if
26:       end if
27:     end for
28:   end for
29: end procedure

```

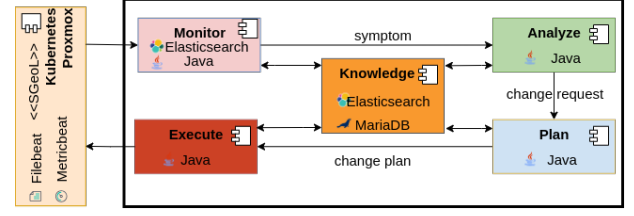
Algorithm 4 Procedure to send a change request

```

1: procedure SENDCHANGEREQUEST( $config$ )
2:    $reviewedCR \leftarrow reviewAllChangeRequests()$ 
3:   for change request  $CR \in reviewedCR$ 
4:     if  $CR.heat = config.heatThreshold$ 
5:        $CR.heat \leftarrow 0$ 
6:        $CR.timestamp \leftarrow getCurrentDate()$ 
7:       if  $CR.itemType = CLUSTER$ 
8:         reset heat values for cluster elements
9:       else if  $CR.itemType = DEPLOYMENT$ 
10:        reset heat values for deployment elements
11:       end if
12:       send change request  $CR$  to Plan phase
13:     end if
14:   end for
15: end procedure

```

it is necessary to configure it, which also slows down the process. For instance, if the implementation uses Kubernetes as a container orchestrator, it must be configured to join an existing Kubernetes cluster. Therefore, vertical scalability is used to increase or decrease the resources of a VM. The change plan is created with actions to increase or decrease VM resources (CPU and RAM). On the other hand, horizontal scaling is applied when the change request is targeting containers. As this virtualization is lighter and has a shorter startup time, adding or removing containers in the environment is a less costly operation when compared to the same operation applied

**Figure 2:** MAPE-K-based auto-scaling architecture

to VMs. Furthermore, this operation should introduce practically no downtime [17].

Algorithm 5 describes the creation of a change plan, which receives the change request created in the *Analyze* phase as a parameter. If the change request is for a cluster or a VM, a change plan will be created using vertical scaling (lines 11-18 and 28-35). Otherwise, a change plan will be created using horizontal scaling (lines 20-26 and 37-45). The amount of resources to be added or removed is always checked for the configured limits. If a change request demands more resources than the configured maximum limit, the plan will be created with the maximum limit value. Finally, the change plan is saved in the *Knowledge* base and sent to the *Execute* phase.

2.4 Execute

The *Execute* phase applies changes to the environment defined in the change plan provided by the *Plan* phase. It must interact with the hypervisor that manages the VMs and the container orchestrator that manages the containers. Operations on the hypervisor or container orchestrator are requested through API calls provided by these components.

Algorithm 6 describes the change plan execution process. This procedure receives the change plan CP generated in the *Plan* phase as a parameter. The plan contains the tasks to be performed in the environment. Each task is logged, allowing administrators or higher-level controllers to track the changes and when they were performed.

By merging application-level metrics with conventional resource usage data, our approach aims to react more quickly and accurately to bottlenecks. For instance, if the platform's internal operation times increase before CPU usage becomes critical, we can initiate scaling immediately to prevent further degradation. In the dynamic and high-stakes environment of smart cities, where essential public services may depend on timely data processing, this earlier response can significantly enhance system reliability.

3 Implementation

We implemented our auto-scaling approach in Java using a modular design that allowed for the decoupling of each MAPE-K phase and the leveraging of specific technologies and tools chosen for their convenience and effective performance. Figure 2 presents an overview of the system architecture, highlighting the technologies used at each MAPE-K phase and their integration with the underlying infrastructure.

The *Monitor* phase requires gathering metrics at three levels: host, container, and platform. We employ two open-source tools

Algorithm 5 Procedure to create a change plan

```

1: procedure CREATECHANGEPLAN(CR)
2:   create change plan CP
3:   CP.CR ← CR
4:   if CR.itemType = VM
5:     CP.target ← CR.vmName
6:   else if CR.itemType = CLUSTER
7:     CP.target ← K.getDeployOfPod(CR.podName)
8:   else CP.target ← CR.deployName
9:   end if
10:  CP.resourceType ← CR.resourceType
11:  if CR.thresholdViolation = UP
12:    if CR.itemType = VM or CR.itemType = CLUSTER
13:      CP.type ← VERTICAL
14:      if CR.scaleQty > K.getMaxVerticalIncQty()
15:        CP.Qty ← K.getMaxVerticalIncQty()
16:      else if CR.scaleQty < K.getMinVerticalIncQty()
17:        CP.Qty ← K.getMinVerticalIncQty()
18:      else
19:        CP.Qty ← CR.scaleQty
20:      end if
21:    else
22:      CP.type ← HORIZONTAL
23:      if CR.scaleQty > K.getMaxHorizontalIncQty()
24:        CP.Qty ← K.getMaxHorizontalIncQty()
25:      else if CR.scaleQty < K.getMinHorizontalIncQty()
26:        CP.Qty ← K.getMinHorizontalIncQty()
27:      else
28:        CP.Qty ← CR.scaleQty
29:      end if
30:    end if
31:  else
32:    if CR.itemType = VM or CR.itemType = CLUSTER
33:      CP.type ← VERTICAL
34:      if CR.scaleQty > K.getMaxVerticalDecQty()
35:        CP.Qty ← K.getMaxVerticalDecQty()
36:      else if CR.scaleQty < K.getMinVerticalDecQty()
37:        CP.Qty ← K.getMinVerticalDecQty()
38:      else
39:        CP.Qty ← CR.scaleQty
40:      end if
41:    else
42:      CP.type ← HORIZONTAL
43:      if CR.scaleQty > K.getMaxHorizontalDecQty()
44:        CP.Qty ← K.getMaxHorizontalDecQty()
45:      else if CR.scaleQty < K.getMinHorizontalDecQty()
46:        CP.Qty ← K.getMinHorizontalDecQty()
47:      else
48:        CP.Qty ← CR.scaleQty
49:      end if
50:    end if
51:  end if
52:  save change plan CP in Knowledge base K
53:  send change plan CP to Execute phase
54: end procedure

```

from the Elasticsearch¹ stack, Metricbeat² and Filebeat.³ Metricbeat

¹<https://www.elastic.co/>

²<https://www.elastic.co/beats/metricbeat>

³<https://www.elastic.co/beats/filebeat>

Algorithm 6 Procedure to execute a change plan

```

1: procedure EXECUTECHANGEPLAN(CP)
2:   create execution task et
3:   et.CP ← CP
4:   et.timestamp ← getCurrentDate()
5:   CO ← getContainerOrchestrator()
6:   if CP.type = VERTICAL
7:     if CP.resourceType = CPU
8:       for vm ∈ CP.target
9:         et.log ← Hypervisor.setCPU(vm, CP.quantity)
10:      end for
11:    else
12:      for vm ∈ CP.targets
13:        et.log ← Hypervisor.setMem(vm, CP.quantity)
14:      end for
15:    end if
16:  else
17:    et.log ← CO.setReplicas(CP.target, CP.quantity)
18:  end if
19:  save execution task et in Knowledge base K
20: end procedure

```

collects infrastructure metrics such as CPU, memory, and disk usage from physical or VMs and container runtimes to reflect the host and container levels. Filebeat gathers logs that include the execution times of internal operations within the smart city platform, as intercepted by AOP. We implemented the *Monitor* phase as a Spring Boot⁴ component that periodically queries Elasticsearch using the Elasticsearch Java API Client. It retrieves host, container, and platform metrics, checks threshold rules, and raises symptoms.

The monitoring agents send data to an Elasticsearch cluster, which serves as the *Knowledge* base for storing time-series metrics. Although we have chosen Elasticsearch for the current implementation, our architecture is flexible enough to accommodate alternative monitoring platforms, e.g., Prometheus. The key requirement is that the monitoring component must expose an API or data access mechanism through which the *Analyze* phase can retrieve relevant metrics and identify threshold violations. This phase operates as another Spring Boot module that processes symptoms provided by the *Monitor* phase.

One of the main contributions of this work is the ability to monitor the application level of the smart city platform without modifying its base code. We accomplish this through AspectJ [10], an AOP-compliant Java extension. We define *aspects* that capture the start and end timestamps of critical internal methods or operations, enabling us to compute the execution time for each operation. The computed times are appended to the application log captured by Filebeat to be forwarded to Elasticsearch. This approach enables us to identify emerging bottlenecks (e.g., a slow database query or a computationally intensive geospatial operation) before CPU usage increases significantly. As a result, our approach can plan and apply scaling actions sooner than a purely infrastructure-based monitoring solution.

Detected symptoms (i.e., threshold violations) are recorded in an instance of the MariaDB⁵ database. While Elasticsearch is best

⁴<https://spring.io/projects/spring-boot>

⁵<https://mariadb.org>

suited for time-series queries, full-text searches, and aggregations over large volumes of logs and metrics, MariaDB stores structured data, such as symptom records, change requests, and change plans. This separation ensures high-performance time-series queries in Elasticsearch, while enabling transactional consistency for change management data in MariaDB.

The *Plan* phase operates as another Spring Boot service, receiving change requests from the *Analyze* phase. It queries the *Knowledge* base to check resource constraints (such as the maximum number of containers or CPU cores) and compiles a list of actions, e.g., adding container replicas, increasing VM memory, among others. The final plan is then stored in the MariaDB database.

During the *Execute* phase, the Executor implements the scaling plan by calling the appropriate APIs: the Kubernetes API for horizontal (container-level) changes and the Proxmox hypervisor API⁶ for vertical (VM-level) adjustments. Regarding vertical scaling, while our approach supports full scale-up, it is subject to a known technical limitation of Proxmox VE itself: dynamic vCPU reduction ('hot-unplug') is not fully supported by the platform or all guest operating systems. This is a platform constraint, not a limitation of our strategy, and memory reduction is generally feasible. Finally, the logs of each execution are recorded back into MariaDB for traceability.

Thanks to the modular design of our approach, each MAPE-K phase can be scaled independently. Our architecture is also flexible, allowing other tools to replace our choices of Elasticsearch and Proxmox without a significant impact.

4 Evaluation

This section presents a series of experiments to evaluate our auto-scaling approach in a real-world smart city platform setting. Section 4.1 outlines the experimental design regarding goals, questions, and three metrics of interest: reaction time, tolerance to processing peaks, and change plan build time. Section 4.2 describes the experimental setup. Section 4.3 presents the parameters we considered in the experiments. Section 4.4 presents and discusses the results of the experiments.

4.1 Goals, Questions, and Metrics

We outlined three main questions (Q1–Q3) to assess different aspects of the MAPE-K loop's effectiveness in detecting and responding to workload changes:

Q1: How much time does the proposed implementation spend to identify a need to change the smart city platform infrastructure?

The *reaction time* (*RT*) corresponds to the time elapsed between the time instant when the first threshold violation occurred (*DFTV*) and the one when a change request is submitted (*CRD*), i.e.:

$$RT = CRD - DFTV \quad (1)$$

Q2: How much does the decision-making mechanism avoid change requests?

The *tolerance to processing peaks* (*TPP*) is the ratio of the number of avoided change requests (*ACR*) and the total of change

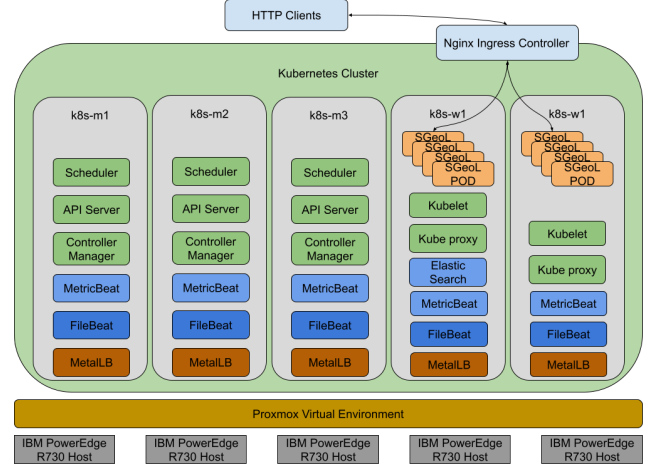


Figure 3: Computational infrastructure used in the experiments

requests sent (*SCR*) and avoided (*ACR*). A change request is avoided if the resource utilization threshold is reached, but the *heat* parameter threshold is not met. A change request is sent when the resource utilization threshold and the *heat* parameter are reached. The *TPP* metric is defined as:

$$TPP = \frac{ACR}{ACR + SCR} \quad (2)$$

Q3: How long does building a change plan from a change request take?

The *change plan build time* (*CPBT*) is the elapsed time between requesting a change request (*CRT*) and building the change plan ready (*CPR*) to be applied, i.e.:

$$CPBT = CPR - CRT \quad (3)$$

Questions Q1 and Q2 are directly related to the *Analyze* phase, while question Q3 is related to the *Plan* phase.

4.2 Experimental Setup

We conducted our experiments on *Smart Geo Layers* (SGeOL) [15], a real-world platform designed to simplify geospatial data management and provide tools for developing diverse smart city applications. SGeOL is deployed as microservices, which are containerized and managed as Pods within a Kubernetes cluster. In turn, this cluster runs on VMs managed by the Proxmox Virtual Environment.

Figure 3 depicts the main components used to build the computational environment for the experiment. Five IBM PowerEdge R730 physical hosts provided computing power with Intel Xeon CPU E5-2630 v3 2.4 GHz, 96 GB of DDR4 RAM, and 1,000 Mbps network cards. The Proxmox Virtual Environment version 7.4-3 managed the physical hosts.

We created seven VMs with Proxmox, five of which were explicitly designated to form a Kubernetes cluster hosting the smart city platform and the Elasticsearch stack. We allocated an additional VM to deploy the implementation of our approach and another

⁶<https://www.proxmox.com/en/proxmox-ve>

Table 1: Parameter configurations used in the experiments

Experiment	Parameter	Configurations
#1	<i>updateFrequency</i>	10, 20, 30, 60, 90 seconds
#2	<i>historyInterval</i>	60, 90, 120, 150, 180 seconds
#3	<i>heatThreshold</i>	1, 2, 4, 8, 16

VM for running Apache JMeter software,⁷ which generated queries simulating user requests. Even though we could have deployed the approach implementation and Apache JMeter on the same Kubernetes cluster, we opted to separate them into different VMs to minimize interference in the experimental environment.

4.3 Experimental Parameters

The experiments studied the *updateFrequency*, *historyInterval*, and *heatThreshold* parameters (see Section 2). Each experiment analyzed these parameters with five different configuration values, as shown in Table 1. We executed twenty runs for each parameter configuration to achieve statistical significance.

When varying a given parameter, the other two remained fixed at their default values. We have set the default values for *updateFrequency*, *historyInterval*, and *heatThreshold* parameters to 30, 120, and 3, respectively.

4.4 Results

We examined each metric regarding the *updateFrequency*, *historyInterval*, and *heatThreshold* parameters across the three experiments. Our aim was to compare the metrics with these parameter settings to gain insights into the impact and influence of these variables on the performance and behavior of the proposed approach. This analysis will provide a comprehensive understanding of how different parameter configurations affect the observed metrics and contribute to optimizing the autonomic scaling process.

Reaction Time (RT) – Q1. The *RT* metric refers to the time required for the proposed approach to detect a need for change in the environment (question Q1). This metric directly relates to sending a change request. A change request can only be generated after an analysis has been performed, so the greater the number of analyses, the greater the chances of generating a change request. Two conditions must be fulfilled to trigger the generation of a change request: (i) the threshold of the monitored item must reach the configured value, and (ii) the heat map must achieve the value set in the *heatThreshold* parameter.

We expect that the higher the value of the *updateFrequency* parameter value, the higher the value of *RT*. Figure 4a illustrates this pattern until *updateFrequency* = 60. The examined data revealed that *updateFrequency* = 90, meaning that the time between two consecutive queries was high enough for variations in workload to occur and not be detected by any query, thus outlining a boundary condition. We concluded that increasing the value of *updateFrequency* increases the *RT* metric. The Pearson correlation coefficient of $r = 0.798$ indicates a high correlation between the average *RT* and the *updateFrequency* parameter.

Figure 4b shows the values obtained for *RT* with variations in the *historyInterval* parameter. The experimental data have not revealed any discernible patterns or relationships between the variation of the *historyInterval* parameter and *RT* (Pearson correlation coefficient $r = -0.092$). We expected this outcome since the *historyInterval* parameter does not impact the amount of generated change requests.

As the value of the *heatThreshold* parameter increases, the time it takes to reach this threshold also increases (Pearson correlation coefficient $r = 0.695$). The metric value increases as *RT* directly relates to sending a change request. Figure 4c shows this behavior.

Tolerance to Processing Peaks (TPP) – Q2. The purpose of the *TPP* metric is to address question Q2. *TPP* indicates the extent to which environmental changes are avoided or delayed during resource usage spikes. The charts presented in Figure 5a, Figure 5b, and Figure 5c show the relationship between *TPP* and the *updateFrequency*, *historyInterval*, and *heatThreshold* parameters during the experiments.

The scaling heat algorithm (see Algorithm 2) is crucial to mitigating frequent variations in the number of change requests generated for the managed environment. Its behavior is significantly influenced by the *heatThreshold* parameter (Pearson correlation coefficient $r = 0.718$). We expected this behavior since the higher the value of the *heatThreshold* parameter, the more violations will occur before a change request is sent. The other parameters do not directly change the number of change requests avoided, so they do not directly influence the *TPP* metric.

Change Plan Build Time (CPBT) – Q3. The *CPBT* metric calculates the time spent to build a change plan (question Q3). Figure 6 shows the relationship between *CPBT* metric and the *updateFrequency*, *historyInterval*, and *heatThreshold* parameters. The time to build a change plan is directly related to the amount of data that must be processed to create the plan. The *updateFrequency* parameter directly influences the number of change requests (Pearson correlation coefficient $r = -0.753$). The lower the value of the *updateFrequency* parameter, the greater the number of requests that must be analyzed to build the change plan and the longer the time required to build this change, as shown in Figure 6a. The other parameters do not strongly influence the number of requests generated. Therefore, they have a short influence on the value of the metric.

While these experiments focused on how parameter tuning impacts metrics, integrating application-level monitoring allows the system to respond to performance degradations before CPU usage escalates. Our results indicate that reactivity depends not only on the frequency and thresholds of monitoring but also on the ability to observe operation times directly within the platform. The approach enables the detection of problems earlier than methods that rely solely on infrastructure metrics.

5 Related Work

The literature on auto-scaling for cloud environments is extensive, with many approaches adopting MAPE-K to facilitate autonomous management. While the existing body of work encompasses numerous strategies, ranging from purely horizontal or vertical scaling to more complex hybrid solutions, a few studies focus on the distinct

⁷<https://jmeter.apache.org>

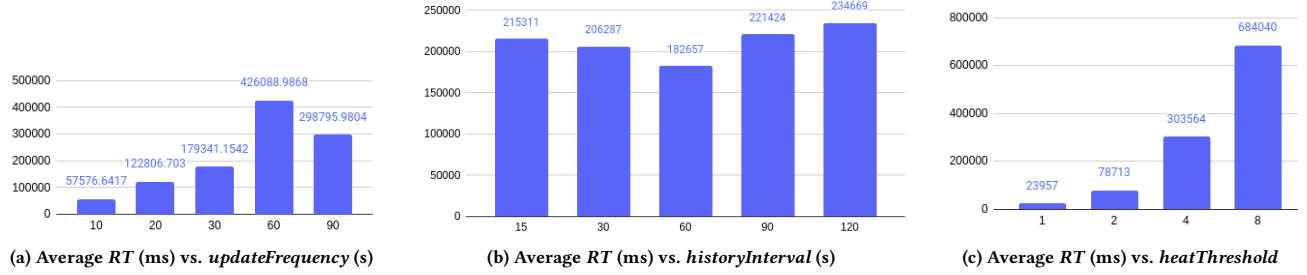


Figure 4: Effect of the analyzed parameters in the reaction time (RT) metric

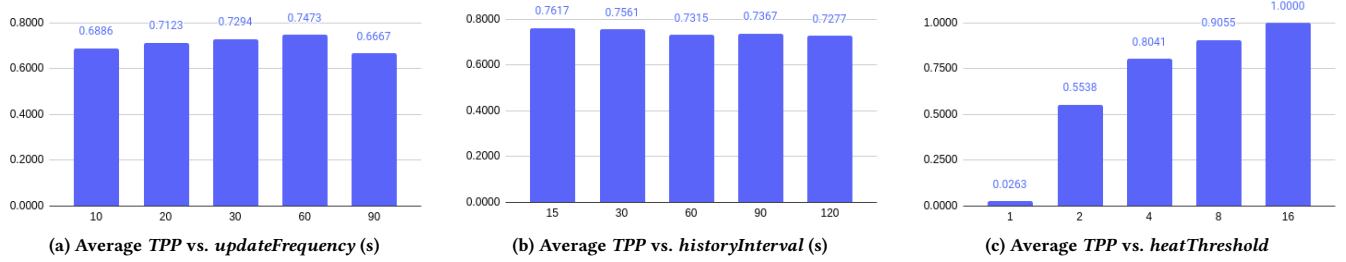


Figure 5: Effect of the analyzed parameters in the tolerance to processing peaks (TPP) metric

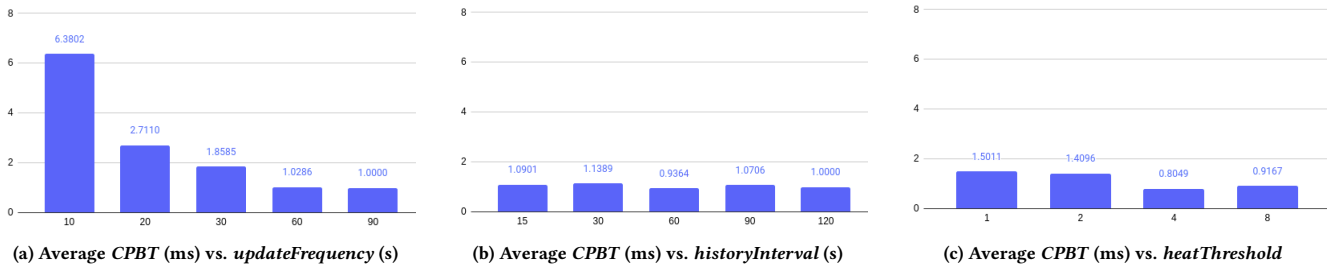


Figure 6: Effect of the analyzed parameters in the change plan building time (CPBT) metric

needs of smart city platforms, which often require fine-grained insights into infrastructure and internal application operations. Table 2 summarizes how various functional requirements are addressed across different works in the literature. In a comparative analysis, our work is uniquely positioned as it offers integrated, multi-level, and non-invasive monitoring, coupled with a comprehensive hybrid scaling strategy specifically tailored for smart city platforms.

MAPE-K-based auto-scaling. Several studies leverage MAPE-K for orchestrating self-adaptation in cloud infrastructures. Oh et al. [14] surveyed five self-adaptive systems using the MAPE-K loop, including two tailored for cloud computing. For example, Hogná [2] deploys self-managing Web applications, deciding when to vertically scale using threshold rules. However, Hogná lacks horizontal elasticity, whereas our approach covers horizontal and vertical scaling. Likewise, TMA [16] implements MAPE-K for containerized systems by using CPU and RAM metrics to detect under- or

over-provisioning in Kubernetes pods. In contrast, we take it a step further by incorporating data from virtual machines and internal platform operations, enabling the earlier detection of performance bottlenecks.

Al-Dhuraibi et al. [1] presented an elastic controller applying MAPE-K to coordinate vertical scaling across VMs and containers. They demonstrate that consolidating these two levels of vertical elasticity can reduce overhead. Our work combines VMs and container monitoring and incorporates application-level information from a smart city platform, highlighting bottlenecks in specific internal operations that might not yet manifest as high CPU usage. This capacity allows us to react proactively, often reducing reaction time.

Vertical and horizontal scaling applied to containers. Ye et al. [18] developed an auto-scaling system for containerized applications. This system features a resource prediction model for proactive resource provisioning and an algorithm that combines

Table 2: Comparative analysis of related work

Requirement	This work	References								
		[18]	[12]	[4]	[1]	[6]	[2]	[16]	[7]	
Non-invasive monitoring	✓					✓				
Multi-level monitoring	✓		✓			×	✓	✓	✓	
CPU monitoring	✓	✓	✓		✓	×	✓	✓	✓	
RAM monitoring	✓	×	✓		✓	×		✓	✓	
Platform monitoring	✓	×	✓		×	×			✓	
Vertical scalability	✓	✓	×	✓	✓	×	✓	×	✓	
Horizontal scalability	✓	✓	✓	✓	×	✓	×	✓	✓	
VM scalability	✓	×	✓	✓	✓	×	✓	×	×	
Container scalability	✓	✓	✓	✓	✓	×	×	✓	✓	
Autonomic capabilities	✓	×	×	×	✓	×	✓	✓	✓	
Reactivity	✓	×	✓	✓	✓	✓	✓	✓	✓	
Proactivity	×	✓	×	✓	×	×	×	×	×	

Blank cells indicate that the study does not present any explicit information about the requirement

vertical and horizontal scaling to prevent service-level agreement violations. While the current implementation primarily focuses on CPU resources, the authors plan to extend support to other resource types as well. Their work distinguishes itself from existing research by integrating vertical and horizontal scaling strategies, offering a more adaptable and efficient solution for containerized environments. This comprehensive approach aims to improve the auto-scaling system's performance and adaptability to workload demands.

Horizontal scaling applied to containers and VMs. Kovács [12] proposed a general interface for implementing programmable scaling policies. These policies utilize metrics from various sources, including infrastructure, applications, and external components. The approach primarily focuses on horizontal scaling for VMs and containers but does not directly address vertical scaling, which is crucial in contexts like smart cities.

Bauer et al. [4] introduced an auto-scaler designed for the coordinated auto-scaling of applications comprising multiple services. The proposed system dynamically monitors the request workload to effectively assess the required resource capacity. It implements vertical and horizontal auto-scaling. The auto-scaler operates through two distinct cycles: (i) a reactive cycle that continuously monitors the application and performs scaling in short intervals, and (ii) a proactive cycle that predicts the demand over longer intervals for future scaling periods. It is worth noting that their auto-scaler relies on an external monitoring component for collecting values like arrival rates, and it is currently focused on request-based applications due to its reliance on respective service demand estimation models.

Monitoring application-level operations. Abranches and Solis [6] proposed a reactive scaling approach that monitors the overall response time behind a load balancer. While effective for specific multi-tier applications, they aggregate all requests into one metric. This coarse-grained view can mask which internal component (e.g., a database or a microservice) is responsible for the delay. In contrast, we track the execution time of individual operations within the smart city platform. This granularity promotes more precise scaling decisions, such as adding resources to a heavily

loaded microservice rather than mindlessly provisioning additional pods for the entire application.

Industry auto-scaling tools. Mainstream tools like the Kubernetes Horizontal Pod Autoscaler (HPA)⁸ and Amazon EKS autoscalers⁹ are widely used. Nonetheless, they exhibit notable limitations. Kubernetes HPA reacts to CPU or memory thresholds at the container level, thus overlooking any internal application metrics and leading to delayed or excessive scaling for more complex, multi-layer services. The Cluster Autoscaler¹⁰ and Karpenter¹¹ do not factor in metrics such as the execution time of platform operations or database queries. Consequently, scaling is triggered solely by resource depletion, such as the number of pending pods.

Although these solutions are suitable for many general workloads, they rarely capture the internal-level signals central to the early detection of bottlenecks. Adopting standard resource metrics and specialized, operation-level data in a smart city environment with intricate application logic and heterogeneous data streams is crucial for timely and efficient scaling. As shown in Table 2, our work is the only solution to combine non-intrusive, multi-level monitoring with both vertical and horizontal scaling under an MAPE-K loop, tailored for smart city workloads.

6 Final Remarks

This paper presented a reactive auto-scaling approach for smart city platforms based on the MAPE-K control loop. Our approach comprises independent components for monitoring the underlying infrastructure and the platform's operations, as well as analyzing the gathered data to determine whether to add or remove resources. We achieve this goal by checking for violations against predefined thresholds and implementing effective adaptations. Our proposal differs from existing work by addressing data monitoring at three levels: host, container, and platform. At the platform level, we leverage data from the internal flow of the platform's operations, captured through AOP in a non-intrusive manner, to inform the scaling decision process. Furthermore, our strategy addresses

⁸<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

⁹<https://aws.amazon.com/eks/>

¹⁰<https://github.com/kubernetes/autoscaler>

¹¹<https://karpenter.sh/>

horizontal and vertical scalability, enabling a more adaptive and efficient resource management in response to the dynamic and diverse demands of smart city environments.

In each phase of the MAPE-K loop, we demonstrated how a heat-based logic, combined with infrastructure- and application-level data, can reduce reaction times while avoiding oscillatory scaling. Although our results validate the benefits of a hybrid (vertical and horizontal) strategy, they also highlight the importance of parameter configuration to match specific workload patterns in a smart city environment. Although our proposal is a reactive strategy, incorporating information from the smart city platform is advantageous. For instance, if the execution time of a critical operation increases, we can adapt resources preemptively rather than waiting for prolonged elevated CPU consumption. This earlier response enhances service continuity and prevents scaling oscillations that could occur if the platform had to wait for infrastructure-level metrics to reach high thresholds.

As future work, we plan to conduct a comprehensive performance comparison against traditional auto-scaling methods, such as the standard Kubernetes HPA, to quantitatively benchmark metrics like reaction time and resource stability. We also aim to enhance the *Analysis* phase by exploring machine learning techniques to anticipate scaling actions and dynamically adjust monitoring thresholds. We advocate that a hybrid approach, combining our application-aware reactive method with proactive ML-based techniques, offers a promising path for enhancing the analysis mechanism.

Acknowledgments

This work is supported by the INCT Intelligent Communications Networks and the Internet of Things - ICoNIoT (<http://iconiot.ic.unicamp.br>), funded by the Brazilian National Council for Scientific and Technological Development - CNPq grant 405940/2022-0 and the Coordination for the Improvement of Higher Education Personnel - CAPES grant 88887.954253/2024-00.

References

- [1] Yahya Al-Dhuraibi, Faiez Zalila, Nabil Djarallah, and Philippe Merle. 2018. Coordinating vertical elasticity of both containers and virtual machines. In *Proceedings of the 8th International Conference on Cloud Computing and Services Science*, Vol. 1. SciTePress, Portugal, 322–329. doi:10.5220/0006652403220329
- [2] Cornel Barna, Hamoun Ghanbari, Marin Litoiu, and Mark Shtern. 2015. Hognu: A platform for self-adaptive applications in cloud environments. In *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. IEEE, USA, 83–87. doi:10.1109/seams.2015.26
- [3] Cornel Barna, Hamzeh Khazaei, Marios Fokaefs, and Marin Litoiu. 2017. Delivering elastic containerized cloud applications to enable DevOps. In *2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. IEEE, USA, 65–75. doi:10.1109/seams.2017.12
- [4] André Bauer, Veronika Lesch, Laurens Versluis, Alexey Ilyushkin, Nikolas Herbst, and Samuel Kounev. 2019. Chamulaton: Coordinated auto-scaling of microservices. In *2019 IEEE 39th International Conference on Distributed Computing Systems*. IEEE, USA, 2015–2025. doi:10.1109/ICDCS.2019.00199
- [5] Tao Chen, Rami Bahsoon, and Xin Yao. 2018. A survey and taxonomy of self-aware and self-adaptive cloud autoscaling systems. *Comput. Surveys* 51, 3 (June 2018). doi:10.1145/3190507
- [6] Marcelo Cerqueira de Abranches and Priscila Solis. 2016. A mechanism of auto elasticity based on response times for cloud computer environments and autossimilar workload. In *2016 XLII Latin American Computing Conference*. IEEE, USA. doi:10.1109/clei.2016.7833403
- [7] Mohamed Hedi Fourati, Soumaya Marzouk, and Mohamed Jmaiel. 2023. Towards microservices-aware autoscaling: A review. In *2023 IEEE Symposium on Computers and Communications*. IEEE, USA, 1080–1083. doi:10.1109/iscs58397.2023.10218213
- [8] IBM. 2005. *An architectural blueprint for Autonomic Computing*. Technical Report. IBM Corporation, USA.
- [9] Jānis Kampars and Krišjānis Pinka. 2017. Auto-scaling and adjustment platform for cloud-based systems. *Environment. Technology. Resources. Proceedings of the International Scientific and Practical Conference 2* (June 2017), 52–57. doi:10.17770/etr2017vol2.2591
- [10] Gregor Kiczales, Erik Hilsdale, Jim Hugunin, Mik Kersten, Jeffrey Palm, and William G. Griswold. 2001. An overview of AspectJ. In *ECOOP 2001 – Object-Oriented Programming*, Jørgen Lindskov Knudsen (Ed.). Lecture Notes in Computer Science, Vol. 2072. Springer Berlin Heidelberg, Germany, 327–354. doi:10.1007/3-540-45337-7_18
- [11] Gregor Kiczales, John Lamping, Anurag Mendhekar, Chris Maeda, Cristina Lopes, Jean-Marc Loingtier, and John Irwin. 1997. Aspect-oriented programming. In *ECOOP'97 – Object-Oriented Programming*, Mehmet Akşit and Satoshi Matsuoka (Eds.). Lecture Notes in Computer Science, Vol. 1241. Springer Berlin Heidelberg, Germany, 220–242. doi:10.1007/bfb0053381
- [12] József Kovács. 2019. Supporting programmable autoscaling rules for containers and virtual machines on clouds. *Journal of Grid Computing* 17, 4 (Aug. 2019), 813–829. doi:10.1007/s10723-019-09488-w
- [13] Matteo Nardelli, Valeria Cardellini, and Emiliano Casalichio. 2018. Multi-level elastic deployment of containerized applications in geo-distributed environments. In *2018 IEEE 6th International Conference on Future Internet of Things and Cloud*. IEEE, USA. doi:10.1109/FiCloud.2018.00009
- [14] Jiyoung Oh, Claudia Raibulet, and Joran Leest. 2023. Analysis of MAPE-K loop in self-adaptive systems for cloud, IoT and CPS. In *Service-Oriented Computing – ICSOC 2022 Workshops*, Javier Troya et al. (Eds.). Lecture Notes in Computer Science, Vol. 13821. Springer Nature Switzerland, Switzerland, 130–141. doi:10.1007/978-3-031-26507-5_11
- [15] Jorge Pereira, Thais Batista, Everton Cavalcante, Arthur Souza, Frederico Lopes, and Nelio Cacho. 2022. A platform for integrating heterogeneous data and developing smart city applications. *Future Generation Computer Systems* 128 (March 2022), 552–566. doi:10.1016/j.future.2021.10.030
- [16] José D'Abruzzo Pereira, Rui Silva, Jorge L. M. Silva Nuno Antunes, Breno de França, Regina Moraes, and Marco Vieira. 2020. A platform to enable self-adaptive cloud applications using trustworthiness properties. In *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. ACM, USA, 71–77. doi:10.1145/3387939.3391608
- [17] Fabiana Rossi, Matteo Nardelli, and Valeria Cardellini. 2019. Horizontal and vertical scaling of container-based applications using reinforcement learning. In *2019 IEEE 12th International Conference on Cloud Computing*. IEEE, USA, 329–338. doi:10.1109/cloud.2019.00061
- [18] Tian Ye, Xue Guangtao, Qian Shiyu, and Li Minglu. 2017. An auto-scaling framework for containerized elastic applications. In *2017 3rd International Conference on Big Data Computing and Communications*. IEEE, USA, 422–430. doi:10.1109/BIGCOM.2017.40